

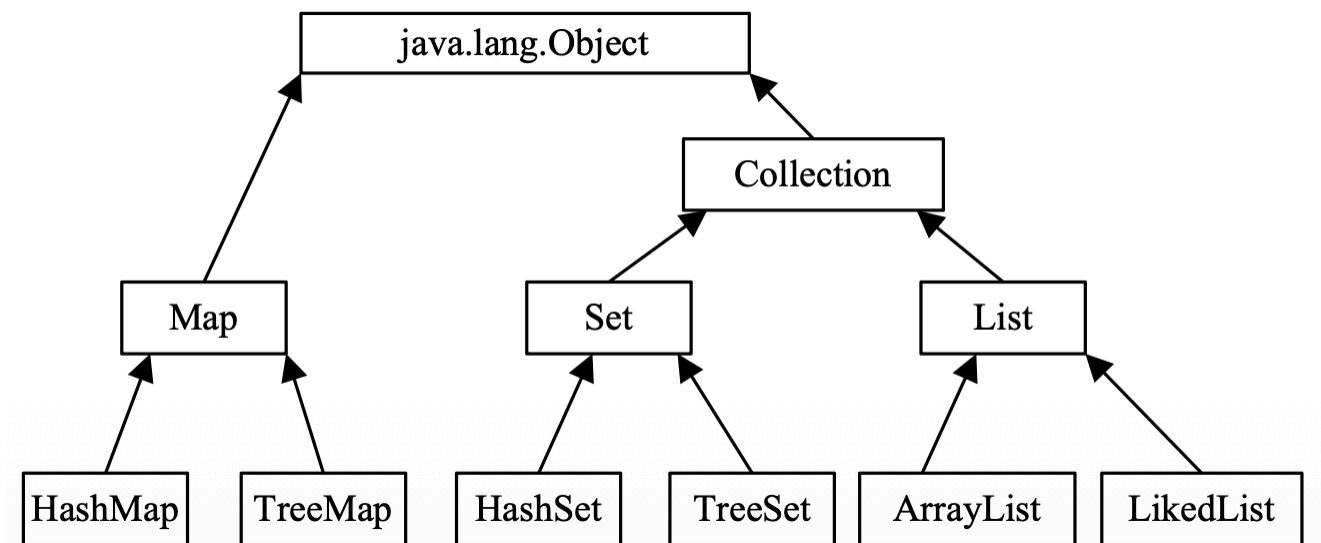
Collection & Map

- [List](#)
 - [ArrayList](#)
 - [LinkedList](#)
- [Set](#)
 - [HashSet](#)
- [Map](#)
 - [HashMap](#)

`java.util` 包中提供了一些集合类，这些集合类又被称为容器。
提到容器难免会想到数组，集合与数组不同之处在于：

- 数组长度固定，而集合长度可变；
- 数组用来存放基本类型的数据和对象的引用，集合只能用来存放对象的引用。

从下图可知，**Java** 集合框架主要包括两种类型的容器，一种是集合（**Collection**），存储一个元素集合；另一种是图（**Map**），存储键值对映射。



List

`List` 定义了一个有序对象的集合，并允许重复元素存在。
`List` 类似于动态数组，其中存放元素的数量，即 `List` 的容量可以随着插入操作自动调整。

ArrayList

`ArrayList` 类是一个可以动态修改的数组，与普通数组的区别就是没有固定的大小限制，提供了相关的添加、删除、修改、遍历等功能。

```
import java.util.ArrayList;
```

```

public class ArrayListTest {
    public static void main(String[] args) {
        ArrayList<String> comps = new ArrayList<String>();

        comps.add("Google");
        comps.add("Apple");
        comps.add("Nvidia");
        // comps = [Google, Apple, Nvidia]

        comps.add(2, "Amazon");
        // comps = [Google, Apple, Amazon, Nvidia]

        comps.remove(3);
        // comps = [Google, Apple, Amazon]

        comps.set(2, "Nvidia"); // 第一个参数为访问的索引位置，第二个为要修改的
值
        // comps = [Google, Apple, Nvidia]

        System.out.println(comps.size());
        // 3

        for (int i = 0; i < comps.size(); i++) {
            System.out.println(comps.get(i));
        }
        // Google
        // Apple
        // Nvidia
    }
}

```

LinkedList

LinkedList 是一种常见的基础数据结构，是一种线性表，但是并不会按线性的顺序存储数据，而是在每一个节点里存到下一个节点的地址。

链表可分为单向链表和双向链表：

- 单向链表的节点包含两个值：当前节点的值和指向后一个节点的链接；
- 双向链表的节点包含三个值：当前节点的值、指向前一个节点的链接和指向后一个节点的链接。

```

import java.util.LinkedList;

public class LinkedListTest {
    public static void main(String[] args) {
        LinkedList<String> comps = new LinkedList<String>();

```

```

comps.add("Amazon");
comps.add("Apple");
comps.add("Nvidia");
// comps: [Amazon, Apple, Nvidia]

comps.addFirst("Google");
// comps: [Google, Amazon, Apple, Nvidia]

comps.add(1, "Meta");
// comps: [Google, Meta, Amazon, Apple, Nvidia]

comps.addLast("OpenAI");
// comps: [Google, Meta, Amazon, Apple, Nvidia, OpenAI]

comps.removeFirst();
// comps: [Meta, Amazon, Apple, Nvidia, OpenAI]

comps.removeLast();
// comps: [Meta, Amazon, Apple, Nvidia]

comps.getFirst();
// Meta

comps.getLast();
// Nvidia
    }
}

```

Set

Set 中的对象不按特定的方式排序，只是简单地把对象加入集合中，但 Set 集合中不能包含重复对象。因此 Set 其实更接近数学意义上的集合。

HashSet

```

import java.util.HashSet;

public class HashSetTest {
    public static void main(String[] args) {
        HashSet<String> sites = new HashSet<String>();

        comps.add("Amazon");
        comps.add("Apple");
        comps.add("Nvidia");
        // comps: [Amazon, Apple, Nvidia]

        comps.add("Amazon"); // 如果添加的元素已经存在，那么集合不变，且不会报错
        // comps: [Amazon, Apple, Nvidia]
    }
}

```

```

        System.out.println(comps.contains("Apple"));
        // true

        comps.remove("Amazon");
        // comps: [Apple, Nvidia]

        System.out.println(comps.size());
        // 2

        comps.clear();
        // comps: []
    }
}

```

Map

Map 接口提供了将 key 映射到 value 的对象。一个映射不能包含重复的 key，每个 key 最多只能映射到一个 value。

HashMap

HashMap 实现了 Map 接口，根据键的 hashCode 值存储数据，具有很快的访问速度，最多允许一条记录的键为 null，不支持线程同步。

HashMap 是无序的，即不会记录插入的顺序。

HashMap 的 key 与 value 类型可以相同也可以不同，可以是 String 类型的 key 和 value，也可以是 Integer 的 key 和 String 类型的 value。

```

import java.util.HashMap;

public class HashMapTest {
    public static void main(String[] args) {
        HashMap<Integer, String> comps = new HashMap<Integer, String>();

        comps.put(1, "Google");
        comps.put(2, "Amazon");
        comps.put(3, "Apple");
        comps.put(4, "Meta");
        // comps: {1=Google, 2=Amazon, 3=Apple, 4=Meta}

        System.out.println(comps.get(3));
        // Apple

        comps.remove(4);
        // comps: {1=Google, 2=Amazon, 3=Apple}

        comps.size();
        // 3
    }
}

```

```
// 输出 key 和 value
for (Integer i : Sites.keySet()) {
    System.out.println("key: " + i + " value: " + Sites.get(i));
}
// key: 1 value: Google
// key: 2 value: Amazon
// key: 3 value: Apple

// 输出 value 值
for (String value: Sites.values()) {
    System.out.print(value + ", ");
}
// Google, Amazon, Apple

comps.clear();
// comps: {}
}
```